DACH catalogue ingestion specification

12 October 2026 · UMAS v3 · JOYN / JOYN_AT / JOYN_CH

Open points

- **Brand search — §6.3.** Product must confirm the end-user search requirement. If retained, confirm API access, readable German labels and capacity at ingestion load; otherwise omit name enrichment.
- **Series publication date — §7.1.** Catalogue/backend must provide the field name, scope and delivery timeline. ARES will index the supplied value.
- **Counters — §9.1.** Confirm the proposed 36-field roster, AVOD/SVOD labels for free/plus, the same windows for views, and explicit nulls for unavailable values.
- **Fingerprint — §8.1.** Finalise source keys, boosts, token format and search analysis; both contributing Loaders must use compatible rules.
- **Schedule encoding and errors — §5.** Finalise tuple escaping/order and the rejection policy for invalid bounds. The availability source and exclusion of raw windows are decided.
- **Rating exceptions — §6.2.** Confirm handling when neither FSK nor FSF is usable, conflicting records, and application of the DE/AT policy to JOYN_CH.
- **UI and stream filters — §7.2.** Confirm Middleware title selection and whether the unselected stream-based lane fields are required by DACH.
- **Configuration and qualification — §10.** Reconcile the workbook with derived fields; assign environment identifiers and qualify the deployed Loader, scheduler, Merge, reject and gRPC paths.

1 Scope and field ownership

Three independent feeds converge on one content index using the same stable UMAS `id`. The supported distribution tenants are **JOYN, JOYN_AT and JOYN_CH**. The architecture supports N feeds; this contract describes three. No source may overwrite another source's business fields.

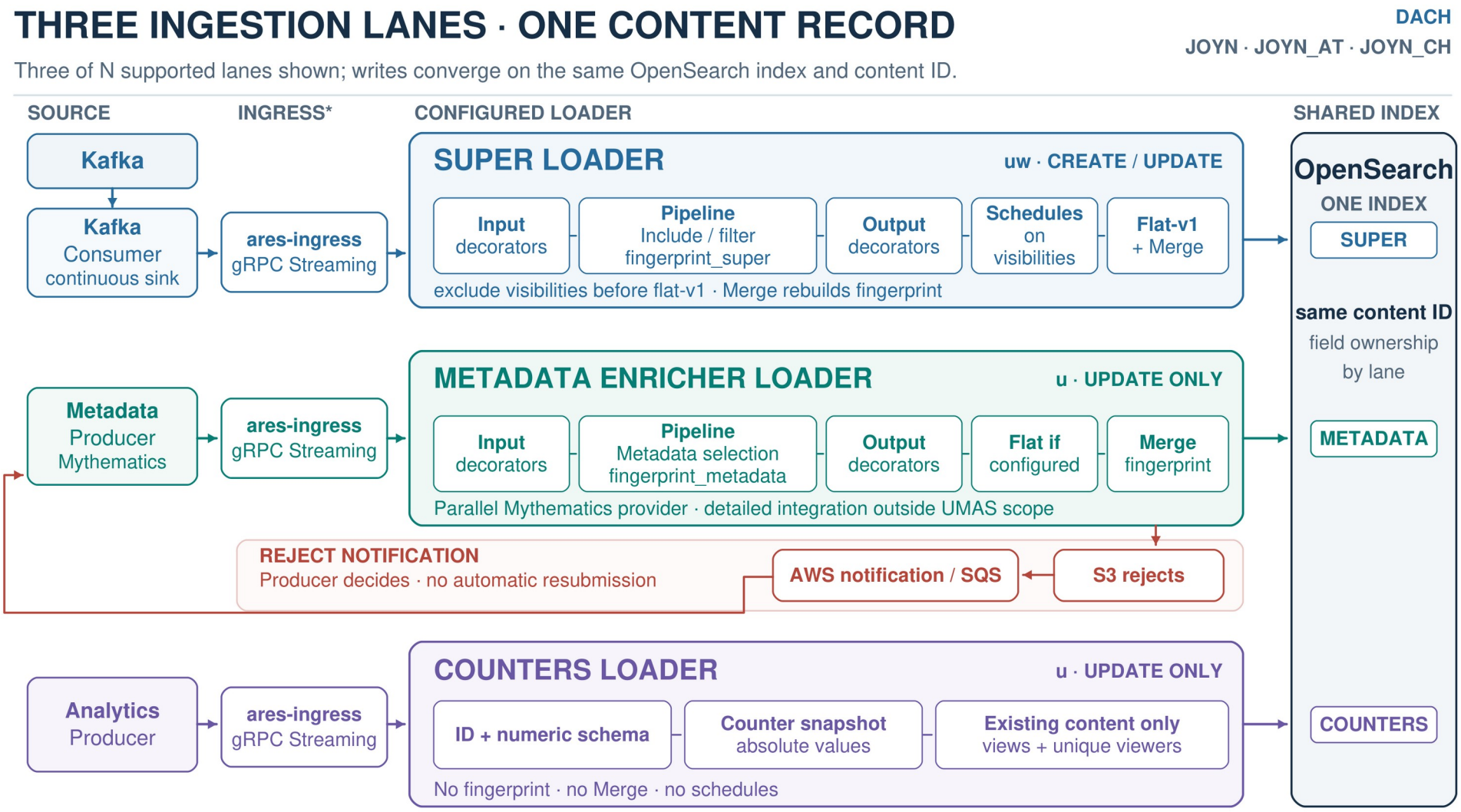
Feed	Action	Owned fields
Super Loader	`uw`	Selected UMAS metadata, dates, availability aggregates and `fingerprint_super`
Metadata Enricher Loader	`u`	Enrichment metadata and `fingerprint_metadata`; detailed provider contract out of scope
Counters Loader	`u`	Precomputed `views_*` and `unique_viewers_*` fields only

Ordinary submissions contain the complete document owned by their source. For counters, this means ID plus the complete counter snapshot; it does not mean resending UMAS metadata. The only producer exception is action `d`, whose payload contains the content ID only.

An upsert merges supplied fields; omitted fields remain stored. Explicit resets are therefore required: `null` for absent scalar dates, `[]` for absent lists and `""` for a cleared fingerprint contribution. Whole-document replacement would erase data from other feeds.

2 Ingestion flow

One index, one content ID, three independently owned sets of metadata. UMAS ingestion is continuous; Metadata and Analytics must not delay it.



*** gRPC Streaming via ares-ingress: ACK validates and queues; indexing is asynchronous.**

Conceptual DACH configuration · three lanes shown of N possible · no deployed state implied

3 Realtime transport and write actions

3.1 gRPC contract

The Kafka Consumer continuously consumes catalogue events and forwards full UMAS documents to `ares-ingress`. Ordinary content updates must not wait for periodic exports. Use the existing bidirectional `AresIngress.stream` RPC with a persistent connection and bounded in-flight messages.

Request field	Meaning
`transactionId`	Producer-generated unique message correlation ID
`token`	Loader access token in each message; determines routing and permitted actions
`action`	Protocol enum: `u = 1`, `uw = 2`, `d = 3`; replacement `w = 0` is not used by this design
`jsonObj`	String containing the JSON document; the two supplied input schemas validate this object
`compressed`	Use `false` for plain JSON; negotiate the supported codec before enabling compression

The response carries `transactionId`, `status` (`OK` or `KO`) and `errorCode`. Correlate each response independently; do not assume completion order. Honour backpressure, bound buffering and drain outstanding responses before closing the stream. Reconnect/replay must retain message correlation and source order for each content ID.

An `OK` acknowledges input validation and queueing. It does **not** confirm an OpenSearch write, schedule persistence or search visibility. Track event-to-ingress and event-to-index latency separately. Protocol basis: the existing ingress guide v1.0, 21 September 2026, §§3–6, and `ares-ingress/protobuf.proto`; verify environment endpoint, TLS, tokens and protocol compatibility before activation.

3.2 Actions and lifecycle

UMAS creation and refresh use `uw`. Deactivation also uses `uw`, sending the full source document with `publishingStatus=INACTIVE`; schedules produce empty availability aggregates. Deletion uses `d` with, for example, `{"id":"SYNTHETIC-001"}`. The Kafka Consumer translates upstream lifecycle events into these actions. Timeslot boundaries are handled internally by ARES schedules.

Metadata and Counters use `u` only: missing content must fail rather than create an incomplete document. Tokens must enforce the allowed actions per feed. A deletion must remove the content and associated schedules; test this combined lifecycle before activation.

4 Import filtering and flat output

The processing order is: input decorators → pipeline Include and list filters → synthesis/source fingerprint → output decorators → schedules/current state → exclude source windows → flat output → output validation → write/Merge.

Include is the first stage inside `ares-pipeline`; input decorators run before it. Include selects structural fields. A list filter removes complete elements and preserves the alignment of their children. For example, `titles[language.code=deu][type=main]` retains German main-title objects. `=` selects, `!=` excludes, `|` expresses alternatives and successive predicates are ANDed.

The returned workbook also selects German genre and keyword names through `genres[].names[language.code=deu]` and `keywords[].names[language.code=deu]`. Keep all CTS tag values together, without category selection. Limit market-bearing visibility/airing lists to the three configured DACH tenants. Preserve future windows and all fields used by schedules until calculation; filtering those windows changes scheduled availability.

4.1 Flat contract and query trade-off

`flatJsonOutput=true` is the agreed UMAS configuration. Root names remain unchanged (`productionYear`); nested leaves become snake_case (`titles[].text → titles_text`). Primitive lists remain lists, including a singleton title. First-level object arrays retain placeholders in the resulting JSON; deeper arrays concatenate. Declare null/empty outputs in `schema.output` so they survive flattening and clear old values.

Flat fields give ARES direct keys for OpenSearch filtering, sorting, aggregations and text search. They avoid nested-query requirements for the selected use cases. They do not preserve same-object relationships in indexed arrays.

For example, source crew entries `[{"name":"Person A","role":"producer"},{"name":"Person B","role":"director"}]` must not satisfy “Person A is the director”. With `crew` mapped as `nested`, the following query correctly returns no match:

```
{"query":{"nested":{"path":"crew","query":{"bool":{"filter":[
  {"term":{"crew.name.keyword":"Person A"}},
  {"term":{"crew.role":"director"}}
]}}}}}
```

Using those conditions on independent `crew_name` and `crew_role` arrays would match incorrectly. The flat design accepts that limitation for crew; date and availability correlations are encoded explicitly below. Raw `visibilities` and their flattened leaves are excluded after scheduling. Retain the workbook-selected flat airing arrays for now; only derived per-tenant dates are authoritative for correlated queries.

5 Availability schedules

5.1 Source and evaluation

Use `visibilities[].timeslots`, for all five UMAS record kinds: Episode, Clip, Movie, Season and Series. Do not use media-object licences as a second availability authority. Each timeslot must retain its parent `distributionTenant`, `type`, `requiredAccessRights` and `brandCodes` until evaluation. The visibility model domain remains AVOD, SVOD, FVOD, RVOD and TVOD.

Configure `applyCurrentState=true`, `skipPastEvents=true`, `excludeFromOutput=true`, active start/end events, and no automatic destroy (`destroyExpired=false`, destroy event inactive). Current state is materialised during ingestion; queued events update it at boundaries. A window is active when `start <= t < end`.

Both start and end scripts receive `($document, $window, $active)` and rebuild all owned arrays from eligible active windows. Return `[true, payload]` including every aggregate. Do not incrementally add/remove values: overlapping windows may still support the same combination. `INACTIVE`, no active windows or an explicit empty window list produces `[]` for every aggregate. Other tenants remain available when only one tenant's window closes.

Normalise an absent owned visibility root to `visibilities: []` under the full-source contract so stale schedules can be cleared. Coalesce equivalent boundary timestamps into one event. Malformed roots and invalid/reversed dates need explicit validation; they are not empty-window defaults. Do not invent a `recommendable` policy. Expiry disables availability; deletion remains an explicit source action.

5.2 Flat aggregate fields

Field mapped as keyword	Example array item	Meaning
`active_distribution_tenants`	`JOYN`	Tenant with eligible active availability
`active_distribution_models`	`JOYN|SVOD`	Tenant and business model
`active_access_rules`	`JOYN|SVOD|[R1,R2]`	Tenant, model and complete rights group
`active_brand_codes`	`JOYN|SVOD|brand-a`	One active brand per tenant/model tuple

These names are the configuration proposal. Preserve the business model in access rules; an empty rights group is valid and differs from no active window. Finalise canonical ordering and delimiter escaping before scripts are activated. These aggregates support editorial queries; they do not implement playback authorisation.

6 Derived metadata

6.1 Broadcast dates

Use `relevantAiringInfo[].datetime` and the `distributionTenant` from the same object. The source contract permits zero or one entry per tenant; both `datetime` and `streamId` are optional. Use the supplied designated airing, which may be past, present or future. The earlier “latest datetime” rule is superseded. Neither deprecated root `broadcastDate` nor visibility broadcast dates is authoritative.

The output decorator uses this complete static mapping:

Source tenant	Index field of type date
`JOYN`	`broadcast_date_joyn`
`JOYN_AT`	`broadcast_date_joyn_at`
`JOYN_CH`	`broadcast_date_joyn_ch`

For every full UMAS update, initialise all three fields to `null`, then populate each from its valid source entry. If JOYN_AT disappears, explicitly emit `broadcast_date_joyn_at: null`; omission would retain its previous value. Use RFC3339 timestamps, normalised to UTC. No epoch-unit inference and no replacement `null_value` in the mapping. Duplicate tenant entries or invalid timestamps must be reported, not silently resolved with max(datetime).

The retained `relevant_airing_info_datetime` and `relevant_airing_info_distribution_tenant` arrays preserve selected data but cannot support same-entry filters. Date predicates and sorting use the three explicit fields above. Adding a tenant requires updating the decorator, output schema and mapping together.

6.2 Age ratings

German and Austrian editors agreed on `ratingSystem=FSK`, otherwise `FSF`, without separate country policies. Select the preferred record in a standard input decorator before projection; retain its `minAge` and `type` together. `type` such as `FSK12` is not the selector `ratingSystem`.

Keep arrays in the flat output. Do not fabricate age zero or silently choose another rating system when neither priority yields a usable record. Conflicting preferred records and missing system identifiers need an explicit handling rule. Confirm that the same agreed policy is applicable to JOYN_CH before enabling that market.

6.3 Brand names for search

Brand codes remain available through the schedule aggregates. Readable Brand names are conditional on Product confirming end-user search by Brand. UI labels alone do not establish that requirement. If it is dropped, no name enrichment is needed.

If confirmed, use the existing API through the same Super Loader's standard input decorator. Proposed lookup: distinct nonempty codes from root `visibilities[].brandCodes[]` for the DACH tenants, across all supplied windows; `getBrandByName(brandName: code)` returns the candidate `title`. Confirm access, German label semantics and capacity. The Brand document's `titleUI` does not prove that API `title` is equivalent.

Write deduplicated readable names to `brand_names` (text with keyword subfield), retain them through Include and add them to search/fingerprint only where configured. No codes means `[]`; a lookup error is not an empty result. API calls add ingestion latency. Brand renames propagate on content reprocessing, not by automatic fan-out.

Do not import a Brand catalogue or BrandChanged topic into ARES and do not create a Brand Loader. If the API cannot meet the requirement, agree readable names supplied upstream. The mapping reserves optional `brand_names`; this does not activate the enrichment.

7 Catalogue and Middleware responsibilities

7.1 Series publication date

The latest eligible episode publication date is calculated by the catalogue/backend, excluding CLIP and EXTRA assets. It is a cross-document calculation. ARES schedules only change the state of their own document from its windows; they do not look up episodes and update another Series document.

The catalogue team has indicated that fields will be supplied; names, final semantics, market scope and timeline remain on hold. When available, the backend must update the Series and send its full source document. ARES indexes the supplied date. No guessed Series field is included in the delivered mapping. Aggregated Series availability may start with the first episode and is not the requested date of the latest episode.

7.2 Titles and stream-based lanes

ARES retains the selected titles as `titles_text`, an array even when it contains one German main title. Editors reference one key. The proposed response boundary is content IDs and a limited field set from ARES, with UI title selection/enrichment in Middleware; confirm it with the catalogue/Middleware owner.

The Brand Management material also describes stream-based lane filters using `relevantAiringInfo.streamId` and `airingLivestreams`. Their applicability to DACH remains unconfirmed and neither was selected in the returned workbook. Do not add them to the index until scope and a tenant-preserving representation are agreed. They do not resolve the Series publication-date requirement.

8 Parallel Metadata enrichment

Mythematics is the parallel Metadata source. Provider implementation and its detailed metadata schema are outside this contract. The **Metadata Enricher Loader** uses the same content ID and UPDATE only, owns its enrichment fields and must not overwrite UMAS or counters. Its source document is complete for its own ownership boundary. It must not block realtime UMAS ingestion.

8.1 Materialised fingerprint Merge

Each pipeline creates its own contribution. Super writes `fingerprint_super`; Metadata writes `fingerprint_metadata`. Configure generic Output Merge on both contributing Loaders to produce `_source.fingerprint` atomically during the write. Super merges its incoming contribution with the stored Metadata contribution; Metadata reverses that ownership. Use the same complete source set and canonical order on both.

A missing stored contribution is empty: the first Super document can produce a valid fingerprint alone. Subsequent writes recompute the combined value rather than appending to its previous value. Every contributing write supplies its own complete contribution; `""` clears it. Do not directly write the merge target in the incoming document. `copy_to` does not materialise that target in `_source`.

Merge is optional and default OFF for compatibility; flat output is independent. Counters do not generate, clear or merge fingerprints. Final source keys, boosts, clean/MD5 token format and search analysis remain configuration decisions. The delivered text mapping is a baseline to qualify with the chosen search rules. Local Merge source code exists; this is not evidence of deployed activation.

8.2 Rejects and notifications

A Metadata UPDATE on missing content must fail. An additional readiness check may use a standard decorator and throw a descriptive error; it is not a hard-coded UMAS rule. Verify that the deployed reject path preserves the original item and reason. Ingress acceptance is not indexing success.

AWS owns notifications from archived rejects to SQS and the Metadata Producer. The producer decides what to do; no automatic resubmission is implied. Disable its Loader retry-after-reject replay in this design. If AWS sends the complete reject payload, delete the S3 object only after confirmed delivery; if it sends a reference, retain the object until it can be retrieved. Handle duplicate notifications outside ARES.

9 Counters Loader

9.1 Metrics and proposed field contract

Analytics supplies two distinct metrics: `unique_viewers` counts distinct viewers in a scope; `views` counts qualifying viewing events, including repeats. One person with three qualifying views gives one unique viewer and three views. ARES receives these values and does not calculate them.

DACH agreed on separate free/plus viewers, per tenant, over 1d, 2d and 7d, initially updated once daily. The concrete schema proposes the same dimensions for views and uses the naming examples AVOD/SVOD for the two segments. The producer must confirm those labels; do not infer them from a content visibility model. Analytics owns identities, event qualification and window boundaries. No lifetime counter is implied.

Dimension	Values in this delivery
Metrics	`unique_viewers`, `views`
Tenants	`JOYN`, `JOYN_AT`, `JOYN_CH`
Proposed segment labels	`AVOD`, `SVOD`
Windows	`1d`, `2d`, `7d`
Names	`<metric>_<tenant>_<segment>_<window>`

The schema enumerates **36 counter fields**. Examples are `unique_viewers_JOYN_SVOD_1d`, `views_JOYN_AT_AVOD_2d` and `views_JOYN_CH_SVOD_7d`. Each is mapped as `long`. The proposed complete snapshot requires the ID and all 36 values; a value is a nonnegative integer, or `null` when unavailable/not applicable. Null clears a previous value and is distinct from measured zero. Confirm this absence convention with the producer; do not invent zeros for unmeasured markets.

Use JSON integers no larger than 9,007,199,254,740,991 so JavaScript processing preserves exact values. A narrower agreed feed roster requires changing its schema and ownership contract explicitly; it must not silently become an arbitrary partial patch.

9.2 Loader profile

Setting	Required configuration
Target and action	Same index and `idField=id`; own Loader/token/queue; token permits `u` only
Validation	Explicit ID + counter allowlist; no additional properties; use the counter schema at both input and output
Pipeline and decorators	Pipeline inactive; no content enrichment or UMAS normalisation
Flat and Merge	`flatJsonOutput=false` (already flat); Output Merge OFF
Schedules	Empty schedule list; never clear source windows from this lane
Write	Absolute assignment to existing content; no increments and no create

The producer sends complete snapshots in order per ID. Repeated current snapshots do not add counts; rolling-window values may decrease. An older snapshot can overwrite a newer one unless upstream ordering/replay prevents it. Daily analytics updates do not change the realtime requirement of the catalogue lane.

Only owned business fields are updated. ARES also writes its standard technical `lastARESLoaderModified` timestamp. Preserve all other metadata, availability and fingerprint fields. UMAS and Metadata must exclude the counter fields even if received upstream. Missing-content failures must be qualified on the deployed Merge-OFF path; do not enable fingerprint Merge just to change error routing.

10 Schemas mapping and activation

10.1 Supporting technical specifications

Specification	Purpose
`content-loader.schema.json`	Technical input JSON Schema for full UMAS documents and ID-only deletion
`counters-loader.schema.json`	Strict input JSON Schema for the complete proposed counter snapshot
`dach-content.mapping.json`	One shared content-index mapping for the three lanes
`content-output-contract.json`	Flat Super output schema for Loader configuration, including nullable resets
`loader-profiles.json`	Reviewable settings and ownership; environment values and scripts still to configure
`contract-roster.json`	Field roster, supported tenants, provenance and pending contract choices

The specifications above define the integration described here. The two input schemas validate the JSON object inside `jsonObj`, not the gRPC envelope. The content schema requires only nonblank `id` and accepts the full source vocabulary; source fields remain optional at this technical boundary. It is not a replacement for UMAS validation upstream, and it cannot prove that a non-delete event is complete. That guarantee belongs to the producer contract.

Input may carry source fields and tenant information that will not be indexed. DACH projection retains only the three configured markets and the selected fields. The shared mapping is strict: no implicit dynamic metadata fields. It excludes raw windows, obsolete broadcast-date aliases and unknown Series fields; it reserves the three fingerprint fields and optional Brand names. Metadata fields must be declared before that out-of-scope feed is activated.

JSON Schemas describe scalar/list/null shapes; OpenSearch maps each indexed value. Arrays use ordinary text/keyword mappings. `productionYear` is a source string and stays keyword. Counter coercion is disabled; dates have explicit formats and no substitute null. Search analysis and index settings remain to qualify.

10.2 Workbook reconciliation

The returned DACH sheet selected 28 of 422 fields. Preserve its selections as evidence, then reconcile configuration with the agreed changes: use derived tenant dates, remove obsolete broadcast-date sources from the index, consume windows for schedules without emitting them, and add declared aggregate/fingerprint/counter fields. All CTS categories remain combined; German title/genre/keyword filters remain effective.

Italy remains a separate legacy-mapping sheet. It does not define DACH names or meanings. The previously delivered unfilled workbook and the returned workbook are not interchangeable. This delivery does not overwrite either spreadsheet.

10.3 Activation checks

Assign Loader IDs, tokens, queues, write/schedule indexes and endpoint. Confirm deployed versions and Bulk/update/script permissions for Merge; these artifacts do not activate an integration.

Test full creation and refresh preserving other lanes; Metadata before/after Super; counter replay/zero/null/lower values; filtering; absent-tenant date resets; empty/future/overlapping windows; INACTIVE; delete with schedule cleanup; and failed writes preserving error reasons.

Declare generated fields in output schemas and verify explicit owned-field resets. Test schedule exceptions: content, schedule rows and backups are not one transaction; failures must not silently leave stale availability.

Close each open item before enabling its capability. Series and conditional Brand enrichment may remain disabled. Qualify index behaviour, API capacity and realtime latency separately from local schema checks.

11 Complete indexed schema dictionary

All **68 mapped fields** are listed below: 32 content/shared fields and 36 counters. JSON types describe the indexed output. The full nested input vocabulary is defined by `content-loader.schema.json` (§10.1).

`[]` denotes a list; `|null` allows null, including placeholders inside parentheses. `datetime` is an RFC3339 string; `date` uses `strict_date_optional_time` (derived dates use UTC). `text + keyword` adds an indexed `.keyword` subfield, not another `_source` property. OpenSearch types apply to each list value.

11.1 Identity and source metadata

Super owns these fields. Optional values depend on record kind; null clears stale scalars. The common id joins all three lanes.

Index field	JSON type	OpenSearch type	Meaning and source
`id`	`string`	`keyword`	Stable UMAS content identifier; the cross-feed join key and Loader idField. Nonblank string.
`type`	`string|null`	`keyword`	UMAS record kind: EPISODE, CLIP, MOVIE, SEASON or SERIES.
`entityType`	`string|null`	`keyword`	Source subtype: FULL_EPISODE for Episode; MOVIE for Movie; CLIP, TRAILER, PROMO or EXTRA for Clip. Not defined for Series/Season.
`publishingStatus`	`string|null`	`keyword`	Source publication state: ACTIVE or INACTIVE. INACTIVE clears all availability aggregates (§5).
`productionCountries`	`string[]|null`	`keyword`	Source production-country codes from productionCountries, e.g. DE or AT; not distribution tenants.
`productionYear`	`string|null`	`keyword`	Source production year as a string. No integer conversion; exact-value filtering uses keyword.
`number`	`integer|null`	`long`	Source episode number for Episode or season number for Season. Its meaning follows type; not a total count.
`duration`	`integer|null`	`long`	Nonnegative duration for Episode, Clip or Movie. Unit is unspecified in the source schema; confirm before duration-based rules.

11.2 Cast and crew

Super projects the source lists. Separate name and role arrays do not preserve a same-person predicate in OpenSearch (§4.1).

Index field	JSON type	OpenSearch type	Meaning and source
`cast_name`	`(string|null)[]|null`	`text + keyword`	Cast person names from cast[].name.
`cast_role`	`(string|null)[]|null`	`keyword`	Cast roles from cast[].role; distinct from crew roles.
`crew_name`	`(string|null)[]|null`	`text + keyword`	Crew person names from crew[].name.
`crew_role`	`(string|null)[]|null`	`keyword`	Crew roles from crew[].role. No controlled vocabulary is declared in the source schema.

11.3 Search labels

Super owns these lists. A single selected value remains a list; absent owned values must be explicitly reset, normally to [].

Index field	JSON type	OpenSearch type	Meaning and source
titles_text	(string|null)[]|null	text + keyword	Text from titles[].text after selecting language.code=deu and type=main. Middleware title presentation remains a separate responsibility (§7.2).
genres_names_text	(string|null)[]|null	text + keyword	German labels from genres[].names[].text; names filtered by language.code=deu.
keywords_names_text	(string|null)[]|null	text + keyword	German labels from keywords[].names[].text; names filtered by language.code=deu.
cts_tags_value	(string|null)[]|null	text + keyword	Values from ctsTags[].value across all categories. Category information is not indexed.
brand_names	(string|null)[]|null	text + keyword	Conditional readable Brand labels resolved from DACH visibility brandCodes by input decorator/API. Reserved pending Product and API confirmation (§6.3).

11.4 Ratings and broadcast dates

Super selects rating records and derives tenant dates before flat output. All three broadcast_date fields are emitted on every refresh, using null when no usable date exists (§6.1). Retained raw airing arrays are not authoritative for tenant/date queries.

Index field	JSON type	OpenSearch type	Meaning and source
age_ratings_min_age	(integer|null)[]|null	long	Minimum-age values from selected ageRatings[].minAge. Prefer FSK, then FSF; exceptions and JOYN_CH applicability remain open (§6.2).
age_ratings_type	(string|null)[]|null	keyword	Source ageRatings[].type from the selected rating records. This is not ratingSystem; FSK/FSF is selected upstream before projection.
relevant_airing_info_datetime	(datetime|null)[]|null	date	Selected relevantAiringInfo[].datetime values. Retained source list, including placeholders for absent dates; use explicit tenant dates for correlation.
relevant_airing_info_distribution_tenant	(string|null)[]|null	keyword	Tenants from relevantAiringInfo[].distributionTenant, restricted to JOYN, JOYN_AT and JOYN_CH. Independent flat list.
broadcast_date_joyn	datetime|null	date	Designated airing datetime for JOYN; nullable scalar derived from the same tenant entry.
broadcast_date_joyn_at	datetime|null	date	Designated airing datetime for JOYN_AT; explicitly null if that tenant or its datetime disappears.
broadcast_date_joyn_ch	datetime|null	date	Designated airing datetime for JOYN_CH; explicitly null if that tenant or its datetime disappears.

11.5 Current availability

Super schedules own these arrays. They are rebuilt from eligible active visibilities, including at ingestion; every array becomes [] when no eligible window remains. Names and tuple encoding are the proposal in §5.2; ordering and escaping require finalisation.

Index field	JSON type	OpenSearch type	Meaning and source
`active_distribution_tenants`	`string[]`	`keyword`	Tenants with eligible active windows, e.g. JOYN.
`active_distribution_models`	`string[]`	`keyword`	Active tenant/model combinations, e.g. `JOYN|SVOD`. Model remains associated with tenant.
`active_access_rules`	`string[]`	`keyword`	Active tenant/model/complete-rights-group tuples, e.g. `JOYN|SVOD|[R1,R2]`. Empty rights groups remain valid; not playback authorisation.
`active_brand_codes`	`string[]`	`keyword`	One active brand code per tenant/model tuple, e.g. `JOYN|SVOD|brand-a`. Does not carry the readable brand name.

11.6 Fingerprints and technical timestamp

The fingerprint fields are stored in `_source`. Source-owned contributions and the shared Merge target have different writers (§8.1); counters never touch them.

Index field	JSON type	OpenSearch type	Meaning and source
`fingerprint_super`	`string`	`text`	Complete Super pipeline fingerprint contribution. Required in Super output; an empty string clears the contribution.
`fingerprint_metadata`	`string`	`text`	Complete Metadata pipeline fingerprint contribution. Absent before enrichment; Merge treats an absent stored contribution as empty.
`fingerprint`	`string`	`text`	Combined contribution materialised atomically by Output Merge on either contributing Loader. Not directly supplied by a producer; token/analyser choices remain open.
`lastARESLoaderModified`	`datetime`	`date`	Ingress envelope timestamp copied into this field by Content Loader. Not a publication date, analytics observation time or index-commit timestamp.

Processing inputs excluded from the index

The Super input also carries `visibilities[].timeslots[].start/end` and their parent `distributionTenant`, `type`, `requiredAccessRights` and `brandCodes`. Schedules consume them before the entire visibility subtree is removed. Rating selection consumes `ageRatings[].ratingSystem`; list filters consume title type and language codes. These processing inputs are not missing index fields: their outputs are listed above. The full source JSON Schema declares their types.

The pending Series publication field and detailed Metadata attributes have no agreed names/contracts and are not invented here. Declare them explicitly in the mapping and relevant schemas before enabling their use. The gRPC envelope fields in §3.1 are transport fields, not content-index properties.

11.7 Unique viewer counters

Counters Loader owns every field in this table and updates existing contents only. All 36 counters across §§11.7–11.8 are required in its complete snapshot, together with id. Values are nonnegative integers up to 9,007,199,254,740,991 or null (unavailable/not applicable); zero is a measured value. OpenSearch long has coercion disabled.

The 36-field roster, AVOD/SVOD segment labels, paired views windows and null convention remain a producer proposal (§9.1). ARES stores upstream counts; it does not compute viewers, views or window boundaries. Each field is scoped to the content identified by id.

Index field	JSON type	OpenSearch type	Meaning and source
`unique_viewers_JOYN_AVOD_1d`	`integer|null`	`long`	Distinct viewers for JOYN, AVOD, 1-day window.
`unique_viewers_JOYN_AVOD_2d`	`integer|null`	`long`	Distinct viewers for JOYN, AVOD, 2-day window.
`unique_viewers_JOYN_AVOD_7d`	`integer|null`	`long`	Distinct viewers for JOYN, AVOD, 7-day window.
`unique_viewers_JOYN_SVOD_1d`	`integer|null`	`long`	Distinct viewers for JOYN, SVOD, 1-day window.
`unique_viewers_JOYN_SVOD_2d`	`integer|null`	`long`	Distinct viewers for JOYN, SVOD, 2-day window.
`unique_viewers_JOYN_SVOD_7d`	`integer|null`	`long`	Distinct viewers for JOYN, SVOD, 7-day window.
`unique_viewers_JOYN_AT_AVOD_1d`	`integer|null`	`long`	Distinct viewers for JOYN_AT, AVOD, 1-day window.
`unique_viewers_JOYN_AT_AVOD_2d`	`integer|null`	`long`	Distinct viewers for JOYN_AT, AVOD, 2-day window.
`unique_viewers_JOYN_AT_AVOD_7d`	`integer|null`	`long`	Distinct viewers for JOYN_AT, AVOD, 7-day window.
`unique_viewers_JOYN_AT_SVOD_1d`	`integer|null`	`long`	Distinct viewers for JOYN_AT, SVOD, 1-day window.
`unique_viewers_JOYN_AT_SVOD_2d`	`integer|null`	`long`	Distinct viewers for JOYN_AT, SVOD, 2-day window.
`unique_viewers_JOYN_AT_SVOD_7d`	`integer|null`	`long`	Distinct viewers for JOYN_AT, SVOD, 7-day window.
`unique_viewers_JOYN_CH_AVOD_1d`	`integer|null`	`long`	Distinct viewers for JOYN_CH, AVOD, 1-day window.
`unique_viewers_JOYN_CH_AVOD_2d`	`integer|null`	`long`	Distinct viewers for JOYN_CH, AVOD, 2-day window.
`unique_viewers_JOYN_CH_AVOD_7d`	`integer|null`	`long`	Distinct viewers for JOYN_CH, AVOD, 7-day window.
`unique_viewers_JOYN_CH_SVOD_1d`	`integer|null`	`long`	Distinct viewers for JOYN_CH, SVOD, 1-day window.
`unique_viewers_JOYN_CH_SVOD_2d`	`integer|null`	`long`	Distinct viewers for JOYN_CH, SVOD, 2-day window.
`unique_viewers_JOYN_CH_SVOD_7d`	`integer|null`	`long`	Distinct viewers for JOYN_CH, SVOD, 7-day window.

11.8 View counters

Counters Loader owns every field in this table and updates existing contents only. All 36 counters across §§11.7–11.8 are required in its complete snapshot, together with id. Values are nonnegative integers up to 9,007,199,254,740,991 or null (unavailable/not applicable); zero is a measured value. OpenSearch long has coercion disabled.

The 36-field roster, AVOD/SVOD segment labels, paired views windows and null convention remain a producer proposal (§9.1). ARES stores upstream counts; it does not compute viewers, views or window boundaries. Each field is scoped to the content identified by id.

Index field	JSON type	OpenSearch type	Meaning and source
views_JOYN_AVOD_1d	integer|null	long	View events incl. repeats; JOYN, AVOD, 1-day window.
views_JOYN_AVOD_2d	integer|null	long	View events incl. repeats; JOYN, AVOD, 2-day window.
views_JOYN_AVOD_7d	integer|null	long	View events incl. repeats; JOYN, AVOD, 7-day window.
views_JOYN_SVOD_1d	integer|null	long	View events incl. repeats; JOYN, SVOD, 1-day window.
views_JOYN_SVOD_2d	integer|null	long	View events incl. repeats; JOYN, SVOD, 2-day window.
views_JOYN_SVOD_7d	integer|null	long	View events incl. repeats; JOYN, SVOD, 7-day window.
views_JOYN_AT_AVOD_1d	integer|null	long	View events incl. repeats; JOYN_AT, AVOD, 1-day window.
views_JOYN_AT_AVOD_2d	integer|null	long	View events incl. repeats; JOYN_AT, AVOD, 2-day window.
views_JOYN_AT_AVOD_7d	integer|null	long	View events incl. repeats; JOYN_AT, AVOD, 7-day window.
views_JOYN_AT_SVOD_1d	integer|null	long	View events incl. repeats; JOYN_AT, SVOD, 1-day window.
views_JOYN_AT_SVOD_2d	integer|null	long	View events incl. repeats; JOYN_AT, SVOD, 2-day window.
views_JOYN_AT_SVOD_7d	integer|null	long	View events incl. repeats; JOYN_AT, SVOD, 7-day window.
views_JOYN_CH_AVOD_1d	integer|null	long	View events incl. repeats; JOYN_CH, AVOD, 1-day window.
views_JOYN_CH_AVOD_2d	integer|null	long	View events incl. repeats; JOYN_CH, AVOD, 2-day window.
views_JOYN_CH_AVOD_7d	integer|null	long	View events incl. repeats; JOYN_CH, AVOD, 7-day window.
views_JOYN_CH_SVOD_1d	integer|null	long	View events incl. repeats; JOYN_CH, SVOD, 1-day window.
views_JOYN_CH_SVOD_2d	integer|null	long	View events incl. repeats; JOYN_CH, SVOD, 2-day window.
views_JOYN_CH_SVOD_7d	integer|null	long	View events incl. repeats; JOYN_CH, SVOD, 7-day window.